Chapter 34 Using the display

This chapter shows how to use the display, for text and graphics output, including control over the associated hardware (LCD window, backlight and contrast).

34.1 Standard facilities

34.1.1 BIOS

For text output, the display is normally accessed and controlled by using the BIOS display service functions, via INT 10H.

For graphics output, INT 10H also provides simple pixel set/read functions, but for better performance, programs normally access the display RAM (B8000..BBFFFH) directly, after selecting a suitable graphics mode and palette using INT 10H.

The standard BIOS video services fall into the following groups:

mode	Set mode	INT 10H, function OH
	Get video state	INT 10H, function FH
scrolling	Scroll up	INT 10H, function 6H
	Scroll down	INT 10H, function 7H
text	Set cursor size	INT 10H, function lH
	Move cursor	INT 10H, function 2H
	Get cursor position	INT 10H, function 3H
	Get cursor address	INT 10H, function 4H
	Set display page	INT 10H, function SH
	Read character and attributes	INT 10H, function 8H
	Write character and attributes	INT 10H, function 9H
	Write character	INT 10H, function AH
	Write character as TTY	INT 10H, function EH
graphics	Set colour palette	INT 10H, function BH
	Write pixel	INT 10H, function CH
	Read pixel	INT 10H, function DH

Hunter 16 implements all these standard IBM PC BIOS (with the exception that function 4 is normally ‘get lightpen

Address'). Refer to Chapter 20 for more details of these functions. Refer to a standard text on programming the IBM PC if you need more general information on using these standard facilities.

The main limitation is that the LCD screen can only display black or white, so colours are displayed as black or white (no grey scales) and text attributes are either ignored or displayed as inverse video. To produce clear displays, limit use of colours and text attributes to those that can be handled properly. (See the description of the inverse video mechanism, later in this chapter.)

The LCD is also not large enough to display the entire CGA contents, but it moves automatically over the CGA screen to show the most recent output, so that should not cause a problem in normal use. To avoid excessive scrolling, try to update the screen sequentially, or limit updating to an area that will fit within the LCD screen. (Or split the screen, as described later in this chapter.)

34.1.2 Character sets

Hunter 16 provides the full IBM character set (see Appendix C), in both text and graphics modes.

34.1.3 RAM

The **Hunter 16** display RAM is an exact emulation of the CGA RAM, so direct access (as is often used for graphics output) is allowed. For more details of the screen RAM organisation, and techniques for accessing it directly, refer to a standard text on programming the IBM PC.

34.1.4 Hardware

To produce non-standard display modes, an application usually accesses the CGA registers directly. Hunter 16 does not implement most of these registers (see section 32.4.1), so only standard modes can be used.

34.2 Extra facilities

34.2.1 BIOS

The **Hunter 16** BIOS provides a wide range of extra display facilities, which can be divided into four groups as follows:

Text	Write string	INT 10H, function 13H
	Select text font	INT 10H, function 73H
	Read text font	INT 10H, function 77H
	Display text attributes as inverse video	INT 10H, function 72H
	Set text attribute table	INT 15H, function 86H
Graphics	Draw line	INT 10H, function 71H
	Draw box	INT 10H, function 71H
	Draw ellipse	INT 10H, function 70H
	Draw circle	INT 10H, function 70H
	Select graphics font	INT 10H, function 60H
	Move cursor to graphics co-ordinates	INT 10H, function 61H
	Read cursor position in graphics co-ordinates	INT 10H, function 62H
Controls	Move LCD window	INT 10H, function 74H
		INT 10H, function 76H
	Set contrast	INT 10H, function 63H
	Set backlight	INT 10H, function 64H
	Set backlight timeout	INT 10H, function 65H
	Toggle screen map mode	INT 10H, function 75H
	Set backlight power up state	INT 15H, function 74H
	Set contrast power up state	INT 1SH, function 75H
Windowing	Split display	INT 10H, function 66H
	Unsplit display	INT 10H, function 66H
	Move window	INT 10H, function 74H

As you can see, most of these facilities are accessed via extensions to INT 10H. Many can also be accessed by the user pressing special key combinations (see section 34.2.3).

34.2.2 Character sets

Hunter 16 provides a range of different character sizes in graphics modes, which can be mixed freely.

The **Hunter 16** BIOS includes the extended character set (ASCII 80..FFH) in graphics modes as well as text modes. An application can replace these standard character definitions by changing the contents of vector 1FH (ie 0007CH..0007FH) to point to its own definitions.

34.2.3 Control keys

The **Hunter 16** display can be controlled by a number of special keys and key sequences, namely:

Moving the LCD	[PAW]plus [Left], [Right], [Up] and [Down] (Looking keys)
LCD contrast	[PAW][Shift] plus [E] or [X]
Backlight	[PAW][L]
LCD map	[PAW][M]
LCD split	[PAW][H] (HOT key)
Inverse video	[PAW][H] (HOT key)

For more details of their use, see Part 2.

34.3 Using the extra facilities

34.3.1 Graphics facilities

In graphics modes, in addition to the normal text output and pixel set/read facilities, the Hunter 16 BIOS provides graphics primitives and a variety of text sizes. You can mix the normal and enhanced facilities quite freely, including mixing text with graphics.

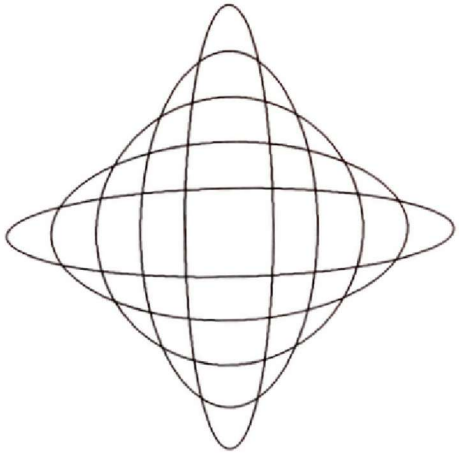
34.3.1.1 Using the graphics primitives

Graphics are drawn using graphics co-ordinates. These depend on the mode, as follows:

Mode	X	Y
4	320	200
5	320	200
6	640	200

Graphics are drawn using INT 10H functions 70H (circles and ellipses) and function 71H (lines, boxes). The graphics origin (0,0) is fixed in the top left corner of the screen.

For example, to draw the following figure:



You could use the following code:

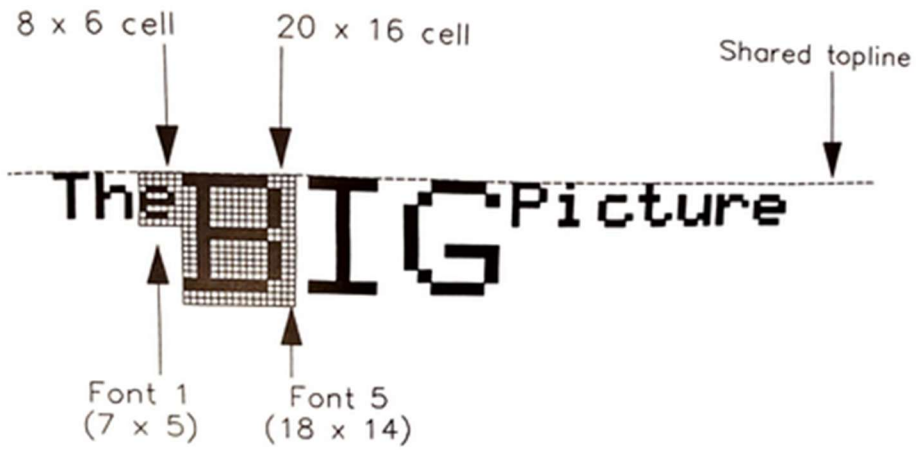
```
;*************************************
;* EXAMPLE: USING GRAPHICS PRIMITIVES *
;*************************************

START:

select mode 6                       ;GRAPHICS MODE
set X MUL to 5                      ;GRAPHICS PARAMETERS

set Y MUL to 1

set AH to 7OH                       ;ELLIPSE/CIRCLE
set DS to data buffer segment       ;POINT TO DATA
set BX to offfset
```

```
DRAW_ LOOP:

call INT 10H                    ;EXECUTE FUNCTION
decrement X MUL
increment Y NUL
if X MUL not 0, jump to DRAW_LOOP
"
"

BUFFER:         DW   320D       ;X CO-0RDINATE OF CENTRE
                DW   100D       ;Y CO-0RDINATE OF CENTRE
                DW   40D        ;RADIUS
                DB   1          ;COLOUR BLACK
X_MUL:          DB   1          ;X MULTIPLIER
Y MUL:          DB   1          ;Y MULITPLIER
```

Note that graphics primitives do not use or change the text cursor position.

34.3.1.2 Using the text fonts

You can change text font (size) at any time when using a graphics mode. When you do, the cursor and character cell size (which is used as the basis for text co-ordinates) adjust automatically, as shown in the following table. The cursor then moves according to the new cell size, so that text is positioned correctly automatically, with simple text output. The cursor does not move when you change text size; if you output text in the new size, it will be immediately to the right of the previous text, sharing the same top line:



The character cell sizes and text co-ordinates for each text font are as follows:

Font no.	Character (Y x X)	Cell (Y x X)	Co-ordinates (rows x cols)
0	7 x7	8 x 8	25 x 80 [40]
1	7 x 5	8 x 6	25 x 106 [53]
2	9 x 7	10 x 8	20 x 80 [40]
3	9 x 14	10 x 16	20 x 40 [20]
4	18 x 7	20 x 8	10 x 80 [40]
5	18 x 14	20 x 16	10x 40 [20]
6	5 x 3	6 x 4	25 x160 [80]

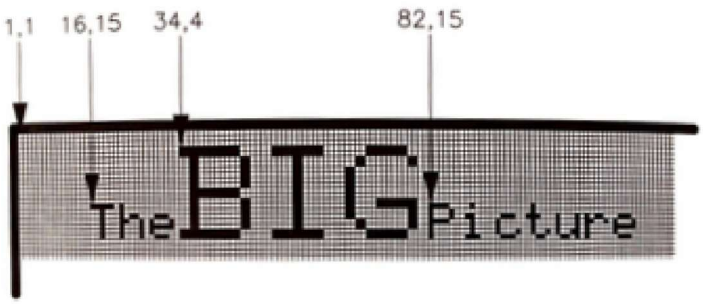
(Figures in square brackets are for the medium resolution modes.)

If you are using a single text size, you simply need to format the output according to the screen width and height for that size as shown by the last column in the table. For example, if you are using font 5 in screen mode 4, format text to 10 lines of 20 columns.

If you are mixing sizes on the screen, you must take more care with positioning to get the best results. The most accurate method is to

position the cursor after changing text size, by using INT 10H function 62H. This function moves the cursor to a character cell whose top left corner is at the specified graphics co-ordinate.

For example, to produce the following result:



You could use the following code:

```
;******************************
;* EXAMPLE: MIXING TEXT SIZES *
;******************************

START:

select mode 6 (function 00H)

set font to 1 (function 60H)
move cursor to (16,15) (function 61H)
write STRING1 by looping (function 0AH)

set font to 5 (function 60H)
move cursor to (34,4) (function 61H)
write STRING2 by looping (function 0AH)

set font to 1 (function 60H)
move cursor to (82,15) (function 61H)
write STRING2 by looping (function 0AH)

return
```

```
;DATA DECLARATIONS

STRING1:        DB   "The"
                LEN1 EQU       $-STRING1 ;LENGTH OF STRING 1

STRING2:        DB   "BIG"
                LEN1 EQU       $-STRING2 ;LENGTH OF STRING 2

STRING3:        DB   "PICTURE"
                LEN1 EQU       $-STRING3 ;LENGTH OF STRING 3
```

You can mix text and graphics freely on the screen. If a character is written over an existing graphic (or text character), the entire character cell is reset to background colour, then the character outline is written in the foreground colour. If a graphic is written over existing text (or another graphic), the foreground part only is written.

34.3.2 Scrolling

The LCD screen normally scrolls automatically to show text input, when a LF is printed on the bottom line of the LCD, or in response to the user pressing the scrolling keys (as described in Part 2).

A program can also position the LCD window directly (ie without user intervention), or split the LCD into two independent windows, which can then be used to view different parts of the CGA screen.

To position the LCD window directly, use INT 10OH function 74H and specify the co-ordinates of the character cell that you want to display in the top left corner of the LCD screen. Remember that even if you have changed the text size (function 60H), the text co-ordinates used for positioning are still based on a 25 line 80 character screen, as described previously (see section 34.3.1.2).

For example, to position the LCD so that it shows the bottom left corner of the screen:

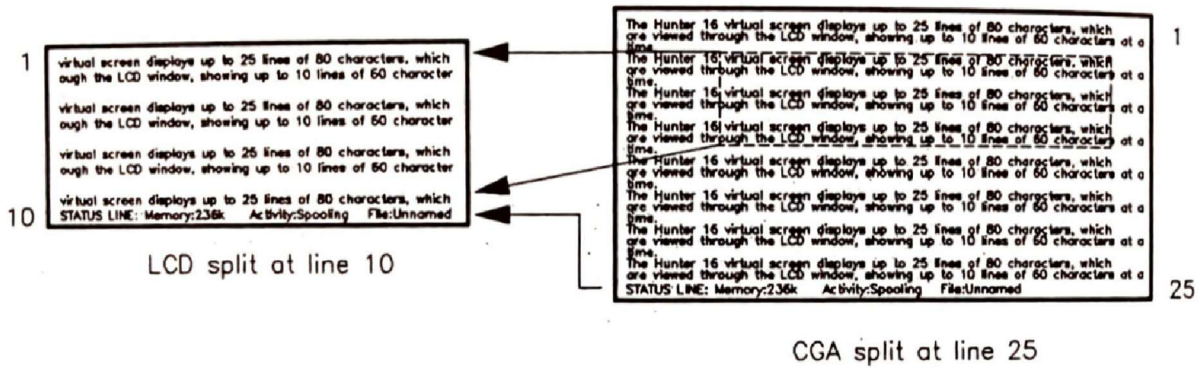
```
set AH to 74H       ;LCD POSITION FUNCTION
set DH to 11H       ;17TH ROW
set DL to 1         ;FIRST COLUMN
INT 10H
```

34.3.3 Windowing

If you are implementing an application that regularly writes to two quite separate areas of the CGA screen, the user will have to scroll the LCD to keep track. To avoid this, you could rewrite the application so that the regular updates are confined to a single area that is small enough to display without scrolling.

If this is not possible, you can minimise the problem by splitting the LCD into two windows, using INT 10H function 66H. For example, suppose that status information is displayed on a single line, the bottom line of the CGA (line number 25).

Create a single-line window for displaying the status line by using function 66H to split the LCD at line 9, and the CGA at line 24.



The top window must show any output in the top part of the CGA, so enable scrolling in it. The bottom window doesn't need to scroll (as it's oly got to show a single line of the CGA), so disable scrolling in it. The code to do this is as follows:

```
set AN to 66H        ;SPLIT WINDOW FUNCTION
set AL to 00000101B ;SPLIT WINDOW, SCROLLING TOP SECTION
set BH to 9          ;SPLIT LCD AT LINE 9
set BL to 24D        ;SPLIT CGA AT LINE 24
INT 10H              ;EXECUTE THE FUNCTION
```

If several lines are used for status information, you might decide to use two LCD lines for them and enable scrolling in the lower half. For example:

```
set AN to 66H       ;SPLIT WINDOW FUNCTION
set AL to 00001101B ;SPLIT WINDOW, ALLOW SCROLLING OF BOTH
SECTIONS
set BH to 6         ;SPLIT LCD AT LINE 6
set BL to 22D       ;SPLIT CGA AT LINE 22
INT 10H             ;EXECUTE THE FUNCTION
```

NOTE: Users can also define or alter the window split for themselves, by using the HOT key, as described in Part 2.

You can use INT 10H function 74H to position the LCD directly; if it is split into windows, this will position whichever window contains the cursor. If the user uses the viewing keys while the LCD is split, this will scroll the window which is defined as scrollable, or both simultaneously, if they are both scrollable.

34.3.4 Using inverse video

The LCD can only display 2 colours (black and white). This may cause problems with applications that rely on other attributes such as high intensity or colour to separate or highlight text.

For many such applications, you can avoid the need for an extensive rewrite by using INT 10H function 72H, to display selected text attributes as inverse video.

For example, suppose an application uses a number of menus, with the current selection indicated by displaying it in high intensity. This will normally be indistinguishable on the **Hunter 16** LCD. Use function 72H to display high intensity text as inverse video, using the following code:

```
set AH to 72H       ;SELECT INVERSE VIDE0 MECHANI SM FUNCTION
set AL to 3         ;SELECT MIGH INTENSI TY TEXT
INT 10H             ;EXECUTE THE FUNCTION
```

If an application uses other attributes (eg background colour or intensity), simply change the AL parameter to select that attribute for display as inverse video.

If you want to display a number of different combinations of text attributes as inverse video, use INT 15H function 86H. This allows you to define the displayed results for all 256 text attributes possible.

These functions have no effect on the contents of the CGA RAM: they simply modify the way that LCD displays these contents. They also have no effect in graphics modes, since these do not support text attributes.

NOTE: Users can also alter use of inverse video for themselves, by using the HOT key, as described in Part 2.

34.3.5 Using the controls

The LCD contrast level and backlight will normally be adjusted by the user pressing special **Hunter 16** keys (see Part 2), but may also be controlled directly by an application using INT 15H functions.

For example, if an application were intended to be used outdoors, it might check the time and turn the backlight on automatically between dusk and dawn, whenever it's displaying a prompt.

Of more general interest is the ability to define a timeout for the backlight (INT 10H, function 65H), so that it will turn off automatically if left on and no keys are pressed for a period. This will help make sure that power is not wasted on the backlight when it is not needed.